

# ■ Guide Ultime : Authentification Google & Firebase/Firestore sur Android (Expo) de A à Z

**Objectif** : Configurer et réussir à 100% l'authentification Google OAuth natif avec Firebase & Firestore dans une application Expo / React Native sur Android, en évitant les rejets silencieux, erreurs **DEVELOPER\_ERROR** et échecs de credential.

## ■ Table des Matières

- [Architecture de l'Authentification Hybride \(Web vs Natif\)](#)
- [Étape 1 : Extraction des Empreintes SHA-1 & SHA-256 via keytool](#)
- [Étape 2 : Configuration dans les Consoles Firebase & Google Cloud](#)
- [Étape 3 : Fichier d'Environnement \(.env\) & Firebase Service](#)
- [Étape 4 : Implémentation du React Context Robuste \(\*\*AuthContext.tsx\*\*\)](#)
- [Étape 5 : Gestion Firestore & Fallback Mode Hors-Ligne](#)
- [■■ Checklist Anti-Erreurs & Troubleshooting](#)

## 1. Architecture de l'Authentification Hybride (Web vs Natif)

Dans une application Expo exécutée à la fois sur le Web et sur Android, le flux d'authentification Google est totalement différent selon la plateforme :

### ■ ÉCOSYSTÈME AUTHENTIFICATION ■

#### ■ PLATEFORME WEB ■ PLATEFORME MOBILE ANDROID ■

■ • Popup Firebase standard ■ • Google Credential Manager ■ ■ • `signInWithPopup(auth)` ■ • Nitro / One-Tap SDK ■ ■ • Utilise le Web Client ID ■ • Jeton ID natif (JWT) ■ ■ • Redirection OAuth directe ■ • Validation via SHA-1 sur Android ■

### Pourquoi la sélection de compte échoue souvent sur Android ?

Sur Android, lorsqu'un utilisateur sélectionne son compte Google dans le dialogue natif :

- Android envoie le nom de package (ex: **com.combistore.app**) et l'**empreinte SHA-1** du fichier APK à Google.
- Si le SHA-1 n'est pas enregistré dans le projet Firebase, Google refuse la validation du jeton (**idToken**).
- Si l'appel **signInWithCredential(auth, credential)** échoue sans bloc try/catch isolé, l'état utilisateur sautera et restera à **null**.

## 2. Étape 1 : Extraction des Empreintes SHA-1 & SHA-256 via keytool

Chaque APK (Debug ou Release) est signé par une clé. Pour que Google autorise votre application Android, vous **DEVEZ** extraire la signature SHA-1 de la keystore.

### Commande PowerShell Windows (avec le JDK Android Studio) :

```
& "C:\Program Files\Android\Android Studio\jbr\bin\keytool.exe" -list -v -keystore  
android/app/debug.keystore -alias androiddebugkey -storepass android -keypass android
```

### Exemple de sortie obtenue :

```
Nom d'alias : androiddebugkey Propriétaire : CN=Android Debug, OU=Android... Empreintes du  
certificat : SHA 1: 5E:8F:16:06:2E:A3:CD:2C:4A:0D:54:78:76:BA:A6:F3:8C:AB:F6:25 SHA 256:  
FA:C6:17:45:DC:09:03:78:6F:B9:ED:E6:2A:96:2B:39:9F:73:48:F0:BB:6F:89:9B:83:32:66:75:91:03:3B:9C
```

■ **Note** : Si vous utilisez une clé de Release personnalisée pour le Play Store, exécutez la même commande sur votre fichier **.jks** ou **.keystore** de production.

## 3. Étape 2 : Configuration dans les Consoles Firebase & Google Cloud

### A. Dans la Console Firebase

- Rendez-vous sur <https://console.firebase.google.com>.
- Sélectionnez ou créez le projet (ex : **combistore-app**).
- Allez dans **Authentification** → **Méthodes de connexion** → Activer **Google**.
- Allez dans **Paramètres du projet** (■) → Onglet **Général**.
- Sous la section **Vos applications**, ajoutez votre application Android :
- Nom de package : **com.combistore.app**
- Cliquez sur **Ajouter une empreinte numérique** (*Add fingerprint*) et collez l'empreinte **SHA-1** :

**5E:8F:16:06:2E:A3:CD:2C:4A:0D:54:78:76:BA:A6:F3:8C:AB:F6:25**

- (*Fortement recommandé*) Ajoutez également l'empreinte **SHA-256**.

### B. Récupérer le Web Client ID

- Toujours dans **Authentification** → **Méthodes de connexion** → **Google**.
- Déroulez jusqu'à **ID client Web**.
- Copiez cette valeur (ex: **130481851392-xxxx.apps.googleusercontent.com**).

## 4. Étape 3 : Fichier d'Environnement (.env) & Firebase Service

## A. Fichier `.env`

```
# Firebase Core Config EXPO_PUBLIC_FIREBASE_API_KEY=AizaSy...
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=combistore-app.firebaseio.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=combistore-app
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=combistore-app.firebaseio.com
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=130481851392
EXPO_PUBLIC_FIREBASE_APP_ID=1:130481851392:web:e894c70... # OAuth Client ID Web (Requis pour
initialiser Google Sign-In) EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID=130481851392-o56691hpmmlhb52tckmm
32dnc9ftmtgn.apps.googleusercontent.com
```

## B. Fichier `src/services/firebase.ts`

Initialisez Firebase avec la persistance adéquate selon la plateforme :

```
import { initializeApp, getApps, getApp, FirebaseApp } from 'firebase/app'; import {
initializeAuth, getAuth, Auth, // @ts-ignore getReactNativePersistence, } from 'firebase/auth';
import { getFirestore, Firestore } from 'firebase/firestore'; import { Platform } from
'react-native'; import AsyncStorage from '@react-native-async-storage/async-storage'; const
firebaseConfig = { apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY, authDomain:
process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN, projectId:
process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID, storageBucket:
process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET, messagingSenderId:
process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID, appId:
process.env.EXPO_PUBLIC_FIREBASE_APP_ID, }; export const isFirebaseConfigured = Boolean(
process.env.EXPO_PUBLIC_FIREBASE_API_KEY && process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID ); let
app: FirebaseApp; let auth: Auth; let db: Firestore; try { if (getApps().length === 0) { app =
initializeApp(firebaseConfig); if (Platform.OS === 'web') { auth = getAuth(app); } else { try {
auth = initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage), }); } catch
{ auth = getAuth(app); } } } else { app = getApp(); auth = getAuth(app); } db =
getFirestore(app); } catch (error) { console.warn('[Firebase] Initialization warning:', error);
// @ts-ignore app = getApps()[0] || null; // @ts-ignore auth = null; // @ts-ignore db = null; }
export { app, auth, db };
```

## 5. Étape 4 : Implémentation du React Context Robuste (`AuthContext.tsx`)

Ce composant garantit que le module natif Nitro Google Sign-In est importé de manière conditionnelle (afin de ne jamais faire planter le bundler Web Metro) et gère la connexion en cas de succès ou de fallback local.

```
import React, { createContext, useContext, useState, useEffect, ReactNode, useCallback } from
'react'; import { Platform } from 'react-native'; import AsyncStorage from
'@react-native-async-storage/async-storage'; import { onAuthStateChanged, signOut as
firebaseSignOut, GoogleAuthProvider, signInWithCredential, signInWithPopup, User as
FirebaseUser, } from 'firebase/auth'; import { auth, isFirebaseConfigured } from
'../services/firebase'; let GoogleOneTapSignIn: any = null; let isSuccessResponse: any = null;
let isNoSavedCredentialFoundResponse: any = null; // Chargement conditionnel dynamique pour
préserver la compatibilité Web if (Platform.OS !== 'web') { try { const nitroGoogle =
require('react-native-nitro-google-signin'); GoogleOneTapSignIn =
nitroGoogle.GoogleOneTapSignIn; isSuccessResponse = nitroGoogle.isSuccessResponse;
isNoSavedCredentialFoundResponse = nitroGoogle.isNoSavedCredentialFoundResponse; } catch (e) {
console.warn('[AuthContext] Could not load react-native-nitro-google-signin:', e); } } export
interface AppUser { uid: string; displayName: string | null; email: string | null; photoURL:
string | null; isAnonymous: boolean; } interface AuthContextType { user: AppUser | null;
loading: boolean; isConfigured: boolean; signInWithGoogle: () => Promise<void>; signOut: () =>
Promise<void>; } const USER_STORAGE_KEY = '@combistore_user'; const AuthContext =
createContext<AuthContextType | undefined>(undefined); export function AuthProvider({ children
```

```

}): { children: ReactNode }) { const [user, setUser] = useState<AppUser | null>(null); const
[loading, setLoading] = useState(true); // Initialisation native One-Tap sur mobile
useEffect(() => { if (Platform.OS !== 'web' && GoogleOneTapSignIn) { const webClientId =
process.env.EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID; if (webClientId) { try {
GoogleOneTapSignIn.configure({ webClientId, offlineAccess: false, }); } catch (err) {
console.warn('[AuthContext] Nitro GoogleOneTapSignIn.configure error:', err); } } }, []); //
Écouteur de l'état d'authentification useEffect(() => { let unsubscribe: (() => void) |
undefined; const initAuth = async () => { try { if (auth && isFirebaseConfigured) { unsubscribe
= onAuthStateChanged(auth, (fbUser: FirebaseUser | null) => { if (fbUser) { const appUser:
AppUser = { uid: fbUser.uid, displayName: fbUser.displayName || 'Utilisateur', email:
fbUser.email, photoURL: fbUser.photoURL, isAnonymous: fbUser.isAnonymous, }; setUser(appUser);
AsyncStorage.setItem(USER_STORAGE_KEY, JSON.stringify(appUser)); } else { setUser(null);
AsyncStorage.removeItem(USER_STORAGE_KEY); } setLoading(false); }); } else { const localUser =
await AsyncStorage.getItem(USER_STORAGE_KEY); if (localUser) { setUser(JSON.parse(localUser));
} setLoading(false); } } catch (err) { console.error('[AuthContext] Init error:', err);
setLoading(false); } }; initAuth(); return () => { if (unsubscribe) unsubscribe(); }, []);
const signInWithGoogle = useCallback(async () => { try { setLoading(true); if (Platform.OS ===
'web') { if (auth && isFirebaseConfigured) { const provider = new GoogleAuthProvider();
provider.setCustomParameters({ prompt: 'select_account' }); await signInWithPopup(auth,
provider); } } else { if (!GoogleOneTapSignIn) { console.warn('[AuthContext] GoogleOneTapSignIn
non disponible'); return; } // 1. Vérification Play Services try { await
GoogleOneTapSignIn.checkPlayServices(true); } catch (playErr) { console.warn('[AuthContext]
Play Services check warning:', playErr); } // 2. Présentation du dialogue natif let response =
await GoogleOneTapSignIn.presentExplicitSignIn().catch(() => null); // 3. Fallback si aucun
identifiant sauvegardé if (!response || (isNoSavedCredentialFoundResponse &&
isNoSavedCredentialFoundResponse(response))) { response = await
GoogleOneTapSignIn.createAccount().catch(() => null); } // 4. Fallback vers la méthode signIn
classique if (!response || (isSuccessResponse && !isSuccessResponse(response))) { response =
await GoogleOneTapSignIn.signIn().catch(() => null); } if (isSuccessResponse &&
isSuccessResponse(response) && response?.data) { const { idToken, user: googleUser } =
response.data; let signedInWithFirebase = false; if (auth && isFirebaseConfigured && idToken) {
try { const credential = GoogleAuthProvider.credential(idToken); await
signInWithCredential(auth, credential); signedInWithFirebase = true; } catch (fbErr) {
console.warn('[AuthContext] Firebase signInWithCredential échoué, fallback local:', fbErr); } }
// Fallback local si Firebase refuse le jeton ou hors-ligne if (!signedInWithFirebase &&
googleUser) { const appUser: AppUser = { uid: googleUser.id || 'google_' + Date.now(),
displayName: googleUser.name || googleUser.givenName || googleUser.email || 'Utilisateur
Google', email: googleUser.email || null, photoURL: googleUser.photo || null, isAnonymous:
false, }; setUser(appUser); await AsyncStorage.setItem(USER_STORAGE_KEY,
JSON.stringify(appUser)); } } } } catch (error: any) { console.error('[AuthContext] Google
Sign-In error:', error); } finally { setLoading(false); } }, []); const signOut =
useCallback(async () => { try { setLoading(true); if (Platform.OS !== 'web' &&
GoogleOneTapSignIn) { await GoogleOneTapSignIn.signOut().catch(() => {}); } if (auth &&
isFirebaseConfigured) { await firebaseSignOut(auth); } await
AsyncStorage.removeItem(USER_STORAGE_KEY); setUser(null); } catch (error) {
console.error('[AuthContext] SignOut error:', error); } finally { setLoading(false); } }, []);
return ( <AuthContext.Provider value={{ user, loading, isConfigured: isFirebaseConfigured,
signInWithGoogle, signOut, }} > {children} </AuthContext.Provider> ); } export function
useAuth() { const context = useContext(AuthContext); if (!context) { throw new Error('useAuth
doit être utilisé dans un AuthProvider'); } return context; }

```

## 6. Étape 5 : Gestion Firestore & Fallback Mode Hors-Ligne

Règles de sécurité Firestore dans la console (Onglet **Firestore Database** → **Règles**) :

```

rules_version = '2'; service cloud.firestore { match /databases/{database}/documents { match
/users/{userId} { allow read, write: if request.auth != null && request.auth.uid == userId; } }

```

```
}
```

## 7. ■■ Checklist Anti-Erreurs & Troubleshooting

| Erreur / Symptôme                                 | Cause Racine                                                                    | Solution                                                                                                              |
|---------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>DEVELOPER_ERROR (Code 10)</b>                  | L'empreinte SHA-1 de la clé d'enregistrement n'est pas dans Firebase.           | Exécuter <code>keytool</code> sur le fichier <code>.keystore</code> et ajouter la clé SHA-1 dans la console Firebase. |
| <b>Crash rouge Web <code>NativeModules</code></b> | Importation statique d'une lib native C++/Nitro sous Web.                       | Charger le module natif de manière conditionnelle via <code>require()</code> si <code>Platform.OS !== 'web'</code> .  |
| <b>Rien ne se passe après le clic compte</b>      | Le jeton d'identité n'a pas pu être validé par Firebase et l'état s'est annulé. | Isoler <code>signInWithCredential</code> dans un try/catch pour forcer le fallback local <code>googleUser</code> .    |
| <b>Erreur de persistance au démarrage</b>         | Persistance Web utilisée sur mobile ou inversement.                             | Utiliser <code>getReactNativePersistence(AsyncStorage)</code> uniquement sur mobile.                                  |

*Guide d'implémentation robuste Firebase & Google Sign-In sur Android.*